

UNIVERSIDAD TECNOLÓGICA DE LOS ANDES

Facultad de Ingeniería de Sistemas



SISTEMAS DE GRAFICACIÓN

Computación Gráfica

Autor: ELISEO PANCORBO DIAZ

Docente: ELVIO TINTAYA ZEGARRA

Cusco, Perú — 2026

RESUMEN

Los sistemas de graficación constituyen el conjunto de herramientas, algoritmos y arquitecturas de hardware y software que permiten la generación, manipulación y visualización de imágenes digitales en un sistema computacional. En el contexto del curso de Computación Gráfica, su estudio resulta fundamental para comprender cómo las representaciones matemáticas abstractas se transforman en imágenes visuales coherentes y de alta calidad. La presente monografía aborda los fundamentos teóricos, los componentes principales, los estándares vigentes y las aplicaciones más relevantes de los sistemas de graficación, analizando el pipeline gráfico, las API de graficación, los modelos de color y las técnicas de renderizado más utilizadas en la industria y la academia.

1. INTRODUCCIÓN

Desde sus orígenes en los años sesenta con los trabajos pioneros de Ivan Sutherland y su programa Sketchpad (1963), la computación gráfica ha evolucionado de forma extraordinaria hasta convertirse en uno de los campos más dinámicos de la ciencia computacional (Foley et al., 1996). Los sistemas de graficación son la infraestructura que hace posible esta evolución: comprenden desde el hardware especializado de las unidades de procesamiento gráfico (GPU) hasta las bibliotecas de programación de alto nivel que permiten a los desarrolladores crear escenas tridimensionales interactivas con gran realismo.

En la actualidad, los sistemas de graficación se aplican en áreas tan diversas como los videojuegos, la simulación científica, la medicina, la arquitectura, el cine de animación y la realidad virtual y aumentada (Shirley y Marschner, 2009). Comprender su funcionamiento interno es indispensable para cualquier profesional de la informática que desee desarrollar aplicaciones visuales de alto rendimiento.

Esta monografía se organiza en cinco secciones principales: fundamentos teóricos, el pipeline gráfico, APIs y estándares de graficación, modelos de color y técnicas de renderizado, y finalmente aplicaciones modernas y perspectivas futuras.

2. FUNDAMENTOS DE LOS SISTEMAS DE GRAFICACIÓN

2.1 Definición y componentes básicos

Un sistema de graficación puede definirse como el conjunto integrado de componentes de hardware y software cuyo propósito es la creación y presentación de imágenes digitales (Zhigilei, 2014). Sus componentes principales incluyen: la unidad central de procesamiento (CPU), la unidad de procesamiento gráfico (GPU), la memoria de video (VRAM), el framebuffer y el dispositivo de visualización.

El framebuffer es una región de memoria que almacena los valores de color de cada píxel que se desplegará en pantalla. La resolución de la imagen y la profundidad de color determinan el tamaño del framebuffer. Por ejemplo, una pantalla Full HD (1920×1080) con color de 24 bits requiere aproximadamente 6,2 MB solo para almacenar un fotograma (Shirley y Marschner, 2009).

2.2 Representación de la información gráfica

Existen dos grandes paradigmas para representar la información visual en un sistema gráfico: la graficación rasterizada (o basada en mapas de bits) y la graficación vectorial. En la representación rasterizada, la imagen se divide en una cuadrícula regular de píxeles, cada uno con un valor de color; este enfoque es el dominante en pantallas de computadora y en el pipeline de renderizado en tiempo real. En la representación vectorial, la imagen se define mediante primitivas geométricas (puntos, líneas, curvas y polígonos) descritas matemáticamente, lo que permite escalarla sin pérdida de calidad (Bézier, 1977, citado en De Berg et al., 2008).

La conversión de primitivas vectoriales a una representación rasterizada se denomina rasterización o escaneo de conversión (scan conversion), y es uno de los procesos centrales del pipeline gráfico moderno (Foley et al., 1996).

3. EL PIPELINE GRÁFICO

3.1 Concepto y etapas generales

El pipeline gráfico (o tubería gráfica) es la secuencia de etapas que un sistema de graficación ejecuta para transformar una descripción geométrica tridimensional de una escena en una imagen bidimensional visualizable en pantalla (Akenine-Möller et al., 2018). Este modelo de procesamiento en cascada permite la paralelización eficiente de tareas y es la base de las GPU modernas.

Las etapas clásicas del pipeline gráfico son: (1) especificación de geometría y atributos, (2) transformaciones geométricas (modelo, vista y proyección), (3) recorte (clipping), (4) rasterización, (5) sombreado de fragmentos y (6) operaciones de framebuffer (prueba de profundidad, mezcla de colores, etc.) (Shirley y Marschner, 2009).

3.2 Transformaciones geométricas

Las transformaciones geométricas son operaciones matemáticas que modifican la posición, orientación y escala de los objetos en la escena. Se representan mediante matrices 4×4 en coordenadas homogéneas, lo que permite combinar traslación, rotación y escala en una única multiplicación matricial (Foley et al., 1996). Las principales transformaciones son:

- Transformación de modelo (Model Transform): lleva cada objeto de su sistema de coordenadas local al espacio del mundo.
- Transformación de vista (View Transform): lleva la escena del espacio del mundo al espacio de la cámara.
- Transformación de proyección (Projection Transform): proyecta la escena tridimensional sobre un plano bidimensional, usando proyección ortogonal o perspectiva.

3.3 Rasterización y sombreado

La rasterización convierte las primitivas geométricas vectoriales (triángulos en la mayoría de los sistemas modernos) en fragmentos de píxeles. Posteriormente, el shader de fragmentos (fragment shader) calcula el color final de cada fragmento en función de la iluminación, las texturas y otros efectos visuales (Akenine-Möller et al., 2018).

Los shaders son programas que se ejecutan directamente en la GPU y permiten efectos visuales complejos como el mapeo de texturas, la reflexión especular, la refracción y las sombras dinámicas. La programabilidad de los shaders, introducida con DirectX 8 y OpenGL 2.0 a inicios de los años 2000, transformó radicalmente las posibilidades expresivas de los sistemas de graficación en tiempo real (Rost et al., 2009).

4. APIs Y ESTÁNDARES DE GRAFICACIÓN

4.1 OpenGL

OpenGL (Open Graphics Library) es la API de graficación multiplataforma más utilizada en la industria y la academia. Desarrollada originalmente por Silicon Graphics Inc. en 1992 y actualmente mantenida por el Grupo Khronos, OpenGL define un conjunto de funciones para la descripción y representación de escenas 2D y 3D (Rost et al., 2009). Su carácter abierto y multiplataforma la convirtió en el estándar de facto para aplicaciones científicas, de ingeniería y de entretenimiento durante más de tres décadas.

La versión 4.6, publicada en 2017, incorpora soporte para renderizado sin servidor de pantalla (headless rendering), sombreado computacional avanzado y compatibilidad mejorada con Vulkan (Khronos Group, 2017).

4.2 Vulkan y las APIs de nueva generación

Vulkan, también desarrollado por el Grupo Khronos y presentado en 2016, representa la siguiente generación de APIs de graficación. A diferencia de OpenGL, Vulkan ofrece acceso de bajo nivel al hardware gráfico, lo que permite al programador gestionar explícitamente la sincronización, la memoria y los comandos de renderizado. Esto reduce la sobrecarga del driver y permite extraer el máximo rendimiento en aplicaciones de tiempo real como los videojuegos modernos (Sellers y Kessenich, 2016).

Microsoft DirectX 12 y Apple Metal siguen una filosofía similar, orientada a maximizar el control del programador sobre el hardware y reducir las capas de abstracción que limitan el rendimiento (Akenine-Möller et al., 2018).

4.3 WebGL y graficación en la web

WebGL es una API de JavaScript basada en OpenGL ES 2.0 que permite la representación gráfica 3D acelerada por hardware directamente en el navegador web, sin necesidad de plugins adicionales (Khronos Group, 2011). Su adopción ha impulsado el desarrollo de aplicaciones de visualización científica, juegos en línea y herramientas de diseño interactivo accesibles desde cualquier dispositivo con navegador moderno.

5. MODELOS DE COLOR Y TÉCNICAS DE RENDERIZADO

5.1 Modelos de color

Los modelos de color son sistemas matemáticos que permiten describir y reproducir los colores de forma precisa y reproducible. Los más utilizados en sistemas de graficación son:

- RGB (Rojo, Verde, Azul): modelo aditivo utilizado en pantallas y dispositivos emisores de luz. Cada canal toma valores entre 0 y 255 (8 bits), lo que permite representar más de 16 millones de colores distintos (Gonzalez y Woods, 2008).
- CMYK (Cian, Magenta, Amarillo, Negro): modelo sustractivo utilizado en impresión.
- HSV/HSL (Matiz, Saturación, Valor/Luminosidad): modelos perceptuales más intuitivos para la selección y edición de colores por parte del usuario.
- CIE XYZ y CIE Lab: modelos basados en la percepción visual humana, utilizados en gestión del color profesional y en perfiles ICC (González y Woods, 2008).

5.2 Técnicas de renderizado

El renderizado (rendering) es el proceso de generar una imagen 2D a partir de una descripción 3D de una escena. Las principales técnicas son:

- Renderizado en tiempo real: basado en el pipeline de rasterización descrito anteriormente. Se ejecuta a velocidades de 30 a 240 fotogramas por segundo, imprescindible en videojuegos y simuladores interactivos (Akenine-Möller et al., 2018).
- Trazado de rayos (Ray Tracing): simula el comportamiento físico de la luz lanzando rayos desde la cámara hacia la escena. Produce imágenes de gran realismo con reflexiones, refracciones y sombras suaves, pero a un costo computacional elevado (Shirley y Marschner,

2009). Con la introducción de las GPU NVIDIA RTX en 2018, el trazado de rayos en tiempo real se ha vuelto cada vez más accesible.

- Radiosidad (Radiosity): técnica basada en el intercambio de energía lumínica entre superficies difusas, adecuada para simular la iluminación global en escenas interiores (Foley et al., 1996).
- Trazado de caminos (Path Tracing): extensión probabilística del trazado de rayos que simula múltiples rebotes de luz para lograr iluminación global fotorrealista. Es el estándar en la industria del cine de animación (Pharr et al., 2016).

6. APLICACIONES Y PERSPECTIVAS FUTURAS

6.1 Aplicaciones contemporáneas

Los sistemas de graficación encuentran aplicación en prácticamente todos los sectores de la actividad humana. En la industria del entretenimiento, los motores de juegos como Unity y Unreal Engine integran pipelines gráficos altamente optimizados con soporte para trazado de rayos en tiempo real, animación de personajes y física basada en la realidad (Epic Games, 2023). En la medicina, la visualización volumétrica de imágenes de tomografía computarizada y resonancia magnética facilita el diagnóstico y la planificación quirúrgica (Gonzalez y Woods, 2008). En la arquitectura y el diseño industrial, las herramientas de CAD y renderizado fotorrealista aceleran el proceso de diseño y permiten la presentación de proyectos con gran fidelidad visual.

6.2 Realidad virtual y aumentada

La realidad virtual (RV) y la realidad aumentada (RA) representan las fronteras más activas de los sistemas de graficación contemporáneos. Estos sistemas imponen requisitos extremadamente exigentes: latencias inferiores a 20 ms para evitar el mareo por movimiento (motion sickness), resoluciones de al menos 90 fotogramas por segundo por ojo y técnicas avanzadas de renderizado foveado adaptativo para reducir la carga computacional (Akenine-Möller et al., 2018). Dispositivos como Meta Quest y Microsoft HoloLens han llevado estas tecnologías al mercado de consumo masivo.

6.3 Inteligencia artificial y graficación

La inteligencia artificial, y en particular las redes neuronales generativas, está transformando profundamente los sistemas de graficación. Técnicas como NeRF (Neural Radiance Fields) permiten sintetizar vistas fotorrealistas de escenas 3D a partir de un conjunto de fotografías 2D (Mildenhall et al., 2020). Los modelos de difusión y las GAN (Redes Generativas Antagónicas) han revolucionado la síntesis de imágenes, texturas y animaciones, reduciendo el tiempo y el costo de producción de contenidos visuales de alta calidad.

7. CONCLUSIONES

Los sistemas de graficación constituyen uno de los pilares fundamentales de la computación moderna. Desde el framebuffer y el pipeline de rasterización hasta las APIs de nueva generación y las técnicas de renderizado fotorrealista, estos sistemas han evolucionado de manera vertiginosa impulsados por la demanda creciente de aplicaciones visuales cada vez más complejas y realistas.

El estudio de sus fundamentos —transformaciones geométricas, modelos de color, algoritmos de sombreado y técnicas de renderizado— proporciona al estudiante de Computación Gráfica las herramientas conceptuales y prácticas para desarrollar aplicaciones visuales de alto rendimiento en cualquier dominio de aplicación.

Las tendencias actuales, marcadas por la convergencia entre el trazado de rayos en tiempo real, la inteligencia artificial y la realidad extendida, auguran un futuro en el que los sistemas de graficación seguirán siendo un área de investigación y desarrollo de primera importancia dentro de la ciencia computacional.

REFERENCIAS BIBLIOGRÁFICAS

- Akenine-Möller, T., Haines, E. y Hoffman, N. (2018). Real-Time Rendering (4.^a ed.). CRC Press.
- De Berg, M., Cheong, O., Van Kreveld, M. y Overmars, M. (2008). Computational Geometry: Algorithms and Applications (3.^a ed.). Springer.
- Epic Games. (2023). Unreal Engine 5 documentation: Lumen global illumination and reflections. <https://docs.unrealengine.com/5.0/en-US/lumen-global-illumination-and-reflections-in-unreal-engine/>
- Foley, J. D., Van Dam, A., Feiner, S. K. y Hughes, J. F. (1996). Computer Graphics: Principles and Practice (2.^a ed. en C). Addison-Wesley Iberoamericana.
- Gonzalez, R. C. y Woods, R. E. (2008). Procesamiento digital de imágenes (3.^a ed.). Pearson Educación.
- Khronos Group. (2011). WebGL specification version 1.0. <https://registry.khronos.org/webgl/specs/latest/1.0/>
- Khronos Group. (2017). OpenGL 4.6 core profile specification. <https://www.khronos.org/opengl/>
- Mildenhall, B., Srinivasan, P. P., Tancik, M., Barron, J. T., Ramamoorthi, R. y Ng, R. (2020). NeRF: Representing scenes as neural radiance fields for view synthesis. Lecture Notes in Computer Science, 12346, 405–421. https://doi.org/10.1007/978-3-030-58452-8_24
- Pharr, M., Jakob, W. y Humphreys, G. (2016). Physically Based Rendering: From Theory to Implementation (3.^a ed.). Morgan Kaufmann.
- Rost, R. J., Licea-Kane, B., Ginsburg, D., Kessenich, J. M., Lichtenbelt, B., Malan, H. y Weiblen, M. (2009). OpenGL Shading Language (3.^a ed.). Addison-Wesley Professional.
- Sellers, G. y Kessenich, J. (2016). Vulkan Programming Guide: The Official Guide to Learning Vulkan. Addison-Wesley Professional.
- Shirley, P. y Marschner, S. (2009). Fundamentals of Computer Graphics (3.^a ed.). CRC Press.

Zhigilei, L. (2014). Introduction to computer graphics and imaging. University of Virginia, Department of Materials Science and Engineering.
<https://people.virginia.edu/~lz2n/mse209/>